# Faceting & Small Multiples

Workshop 2 — Module 5

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

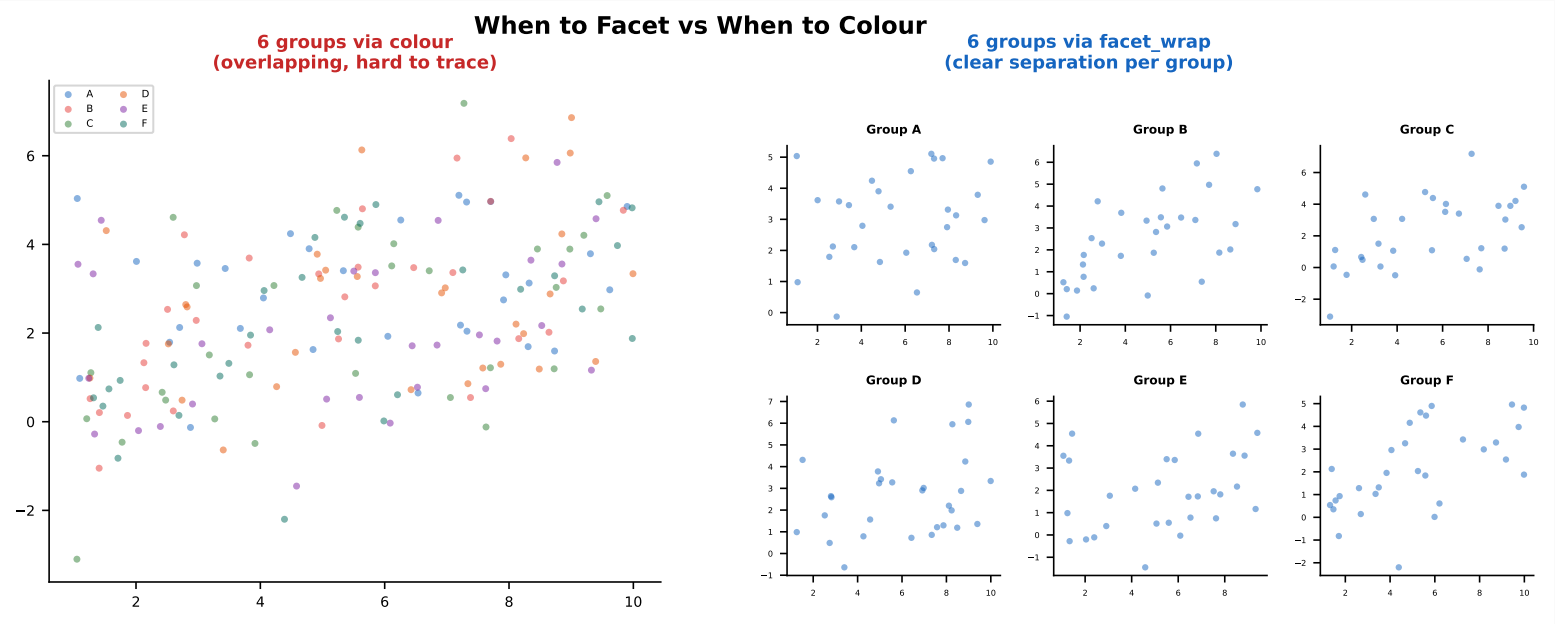
# Module Roadmap

1. Tufte's small multiples revisited via the grammar
2. `facet_wrap()`: one variable, wrapped grid
3. `facet_grid()`: two variables, rows $\times$ columns
4. Fixed vs free scales: the critical trade-off
5. Facet vs colour: when to use which
6. Faceting in Python: plotnine, seaborn FacetGrid, matplotlib

**Learning Outcome**
You will use faceting to produce Tufte-quality small multiples that separate groups into independent panels while preserving shared axes for fair comparison.

# Small Multiples via the Grammar

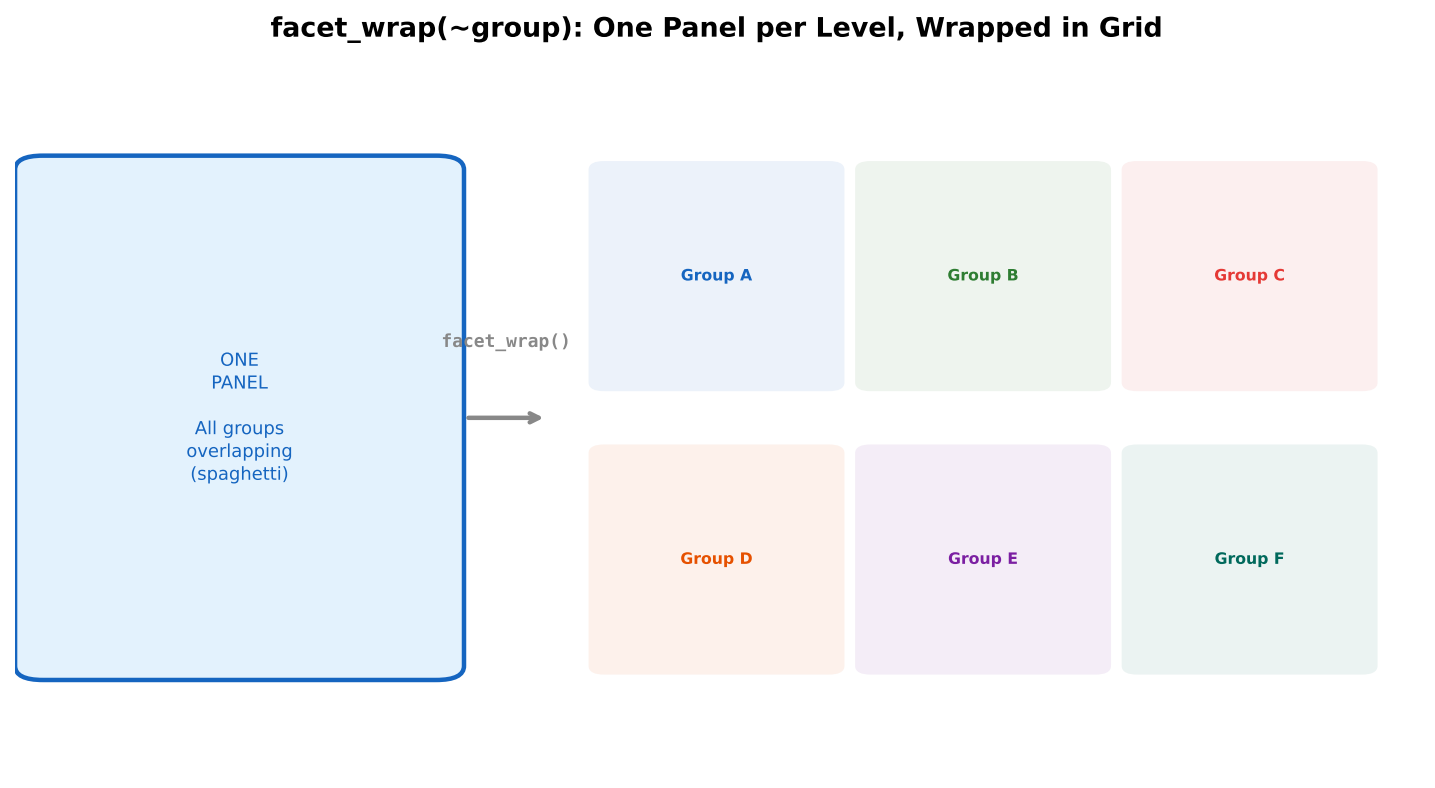
# From Spaghetti to Small Multiples




**Pro-Tip**

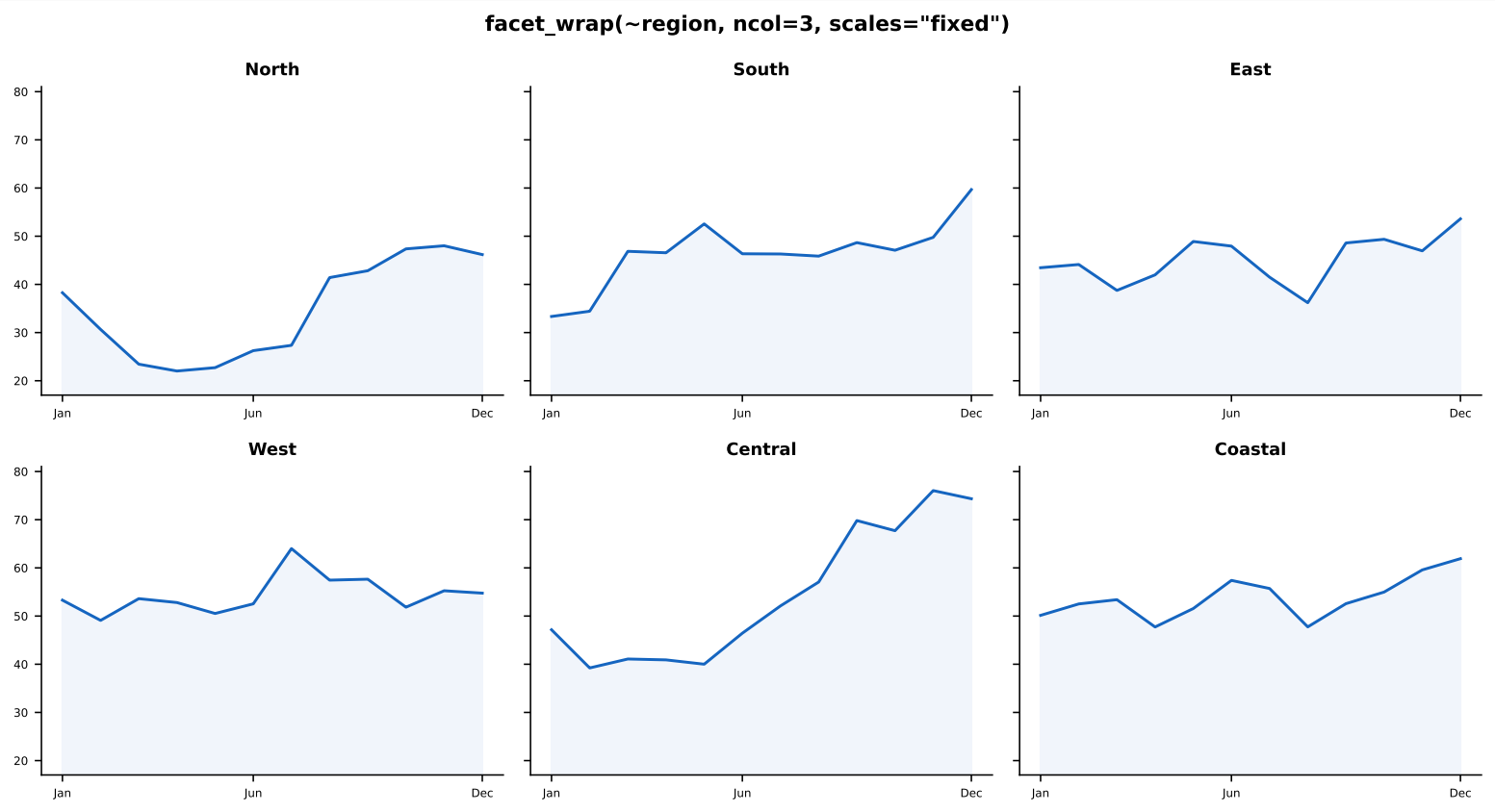
Colour works for $\leq 3$ groups. For 4+ groups, faceting produces cleaner separation by exploiting Gestalt proximity (W01-M03) instead of hue discrimination.

# facet_wrap: Concept



# facet_wrap()

# facet_wrap Demo: Six Regions

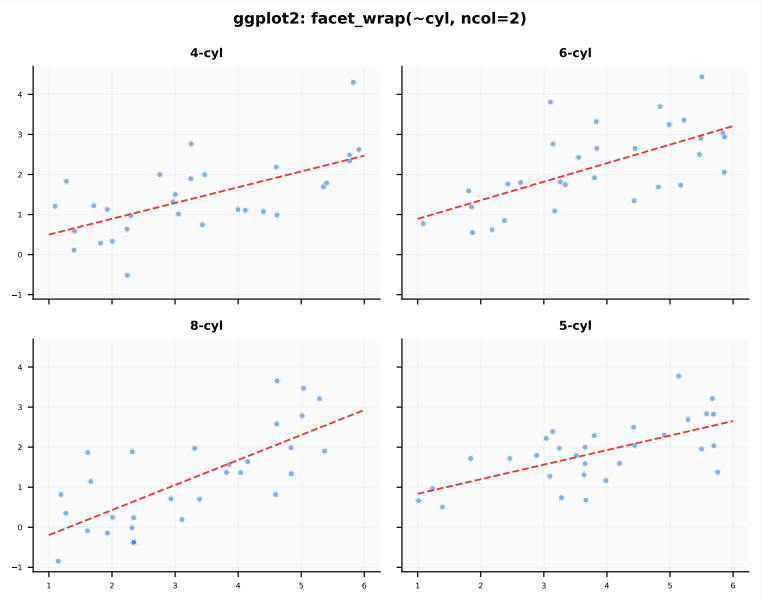


# facet_wrap in R

```r
ggplot(df, aes(x = month,
               y = revenue)) +
  geom_line(color = "#1565C0",
            linewidth = 1) +
  geom_area(fill = "#1565C0",
            alpha = 0.06) +

  # One panel per region
  facet_wrap(
    ~ region,
    ncol = 3,          # 3 columns
    scales = "fixed"   # shared y
  ) +

  scale_x_continuous(
    breaks = c(1, 6, 12),
    labels = c("Jan","Jun","Dec"))
        +
  theme_minimal(base_size = 9) +
  theme(strip.text =
    element_text(face = "bold")) +
  labs(title = "Revenue by Region",
       x = NULL, y = "Revenue ($K)"
           )
```




Key arguments: `ncol` / `nrow` control the grid layout. `scales = "fixed"` (default) ensures fair comparison. The strip text above each panel shows the faceting variable's level.

# facet_wrap in Python

## plotnine (ggplot2 clone)

```
from plotnine import *

(ggplot(df, aes(x="month",
                y="revenue"))
 + geom_line(color="#1565C0")
 + facet_wrap("~region", ncol=3)
 + theme_minimal())
```

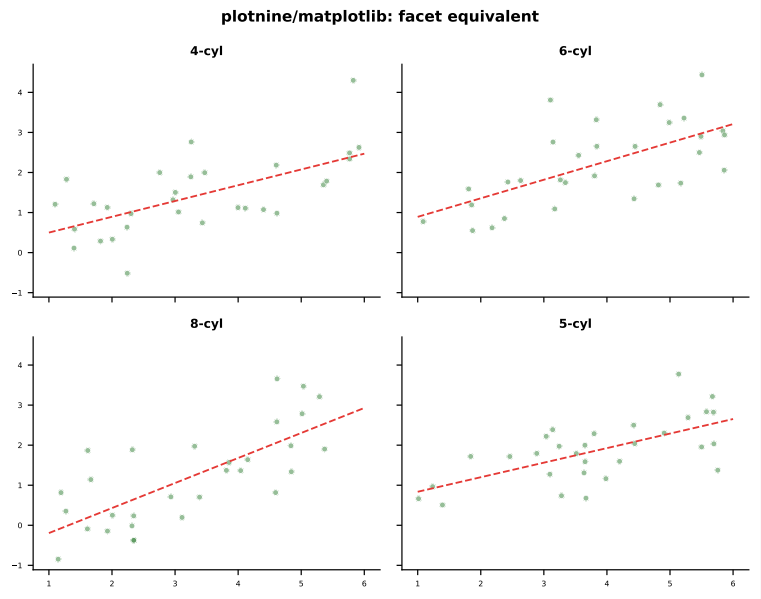
## seaborn FacetGrid

```
import seaborn as sns

g = sns.FacetGrid(df,
    col="region", col_wrap=3,
    sharey=True, height=3)
g.map(plt.plot, "month","revenue",
      color="#1565C0")
g.set_titles("{col_name}")
```

## matplotlib (manual)

```
fig, axes = plt.subplots(
```

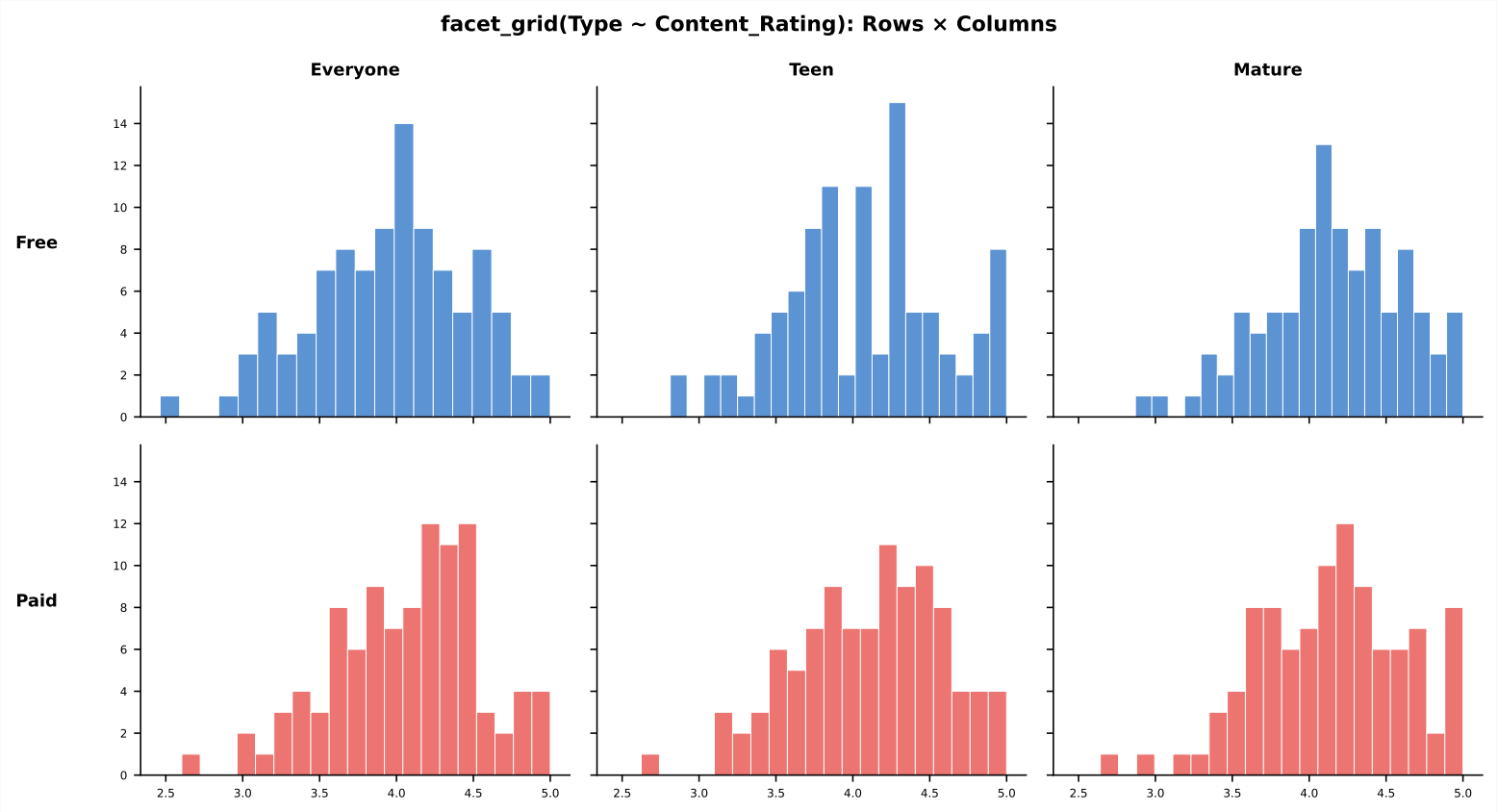



Three Python approaches for faceting, ordered by automation level:

1. `plotnine`: identical to ggplot2 syntax
2. `seaborn.FacetGrid`: map function to panels
3. `plt.subplots`: full manual control

# facet_grid()

# facet_grid: Rows × Columns




**Data Insight**

`facet_grid(Type ~ Content_Rating)` creates a matrix: rows = Type (Free, Paid), columns = Content Rating (Everyone, Teen, Mature). Every cell shows the same chart type for that combination. Use `facet_grid` when you have two categorical variables; use `facet_wrap` when you have one.

# facet_grid in Code

## R

```r
ggplot(apps, aes(x = Rating)) +
  geom_histogram(bins = 20,
    fill = "#1565C0",
    color = "white") +

  # rows = Type, cols = Content
      Rating
  facet_grid(
    Type ~ `Content Rating`,
    scales = "fixed") +

  theme_minimal(base_size = 9) +
  labs(title = "Rating by Type
       x Content Rating")
```

## Python (plotnine)

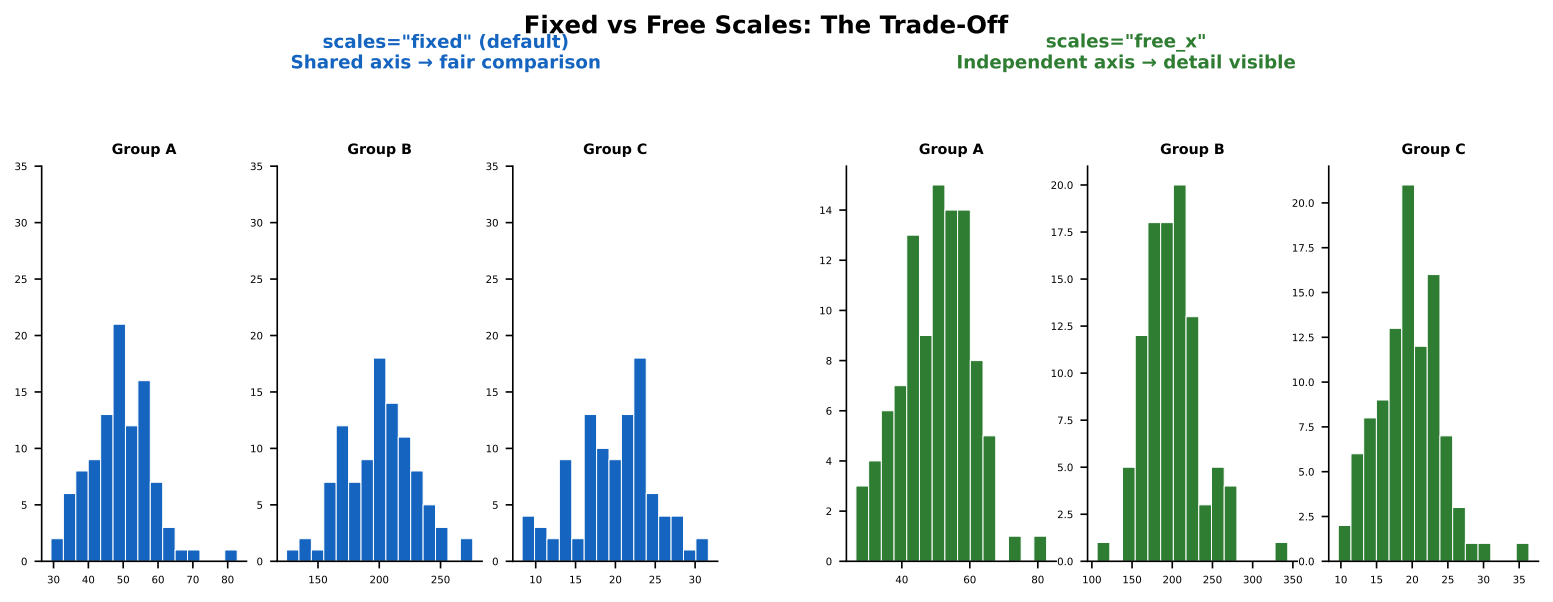
```python
(ggplot(apps, aes(x="Rating"))
 + geom_histogram(bins=20,
     fill="#1565C0", color="white
         ")
 + facet_grid("Type~
     Content_Rating")
 + theme_minimal())
```

## seaborn

```python
g = sns.FacetGrid(apps,
    row="Type",
    col="Content_Rating",
    sharey=True, sharex=True)
g.map(plt.hist, "Rating",
      bins=20, color="#1565C0")
```

# Fixed vs Free Scales

# The Critical Trade-Off



# Scale Options

| Setting | Effect |
|---|---|
| `scales = "fixed"` | All panels share x and y (default). Best for fair comparison across groups. |
| `scales = "free_x"` | Each panel has its own x range. Y stays shared. |
| `scales = "free_y"` | Each panel has its own y range. X stays shared. |
| `scales = "free"` | Both axes independent. Best for showing within-group detail. |

**Pro-Tip**

**Default to fixed scales** for comparison tasks. Use free scales only when groups have vastly different ranges (e.g., countries with populations from 1K to 1B). When using free scales, warn the reader with an annotation: "Note: axes differ across panels."

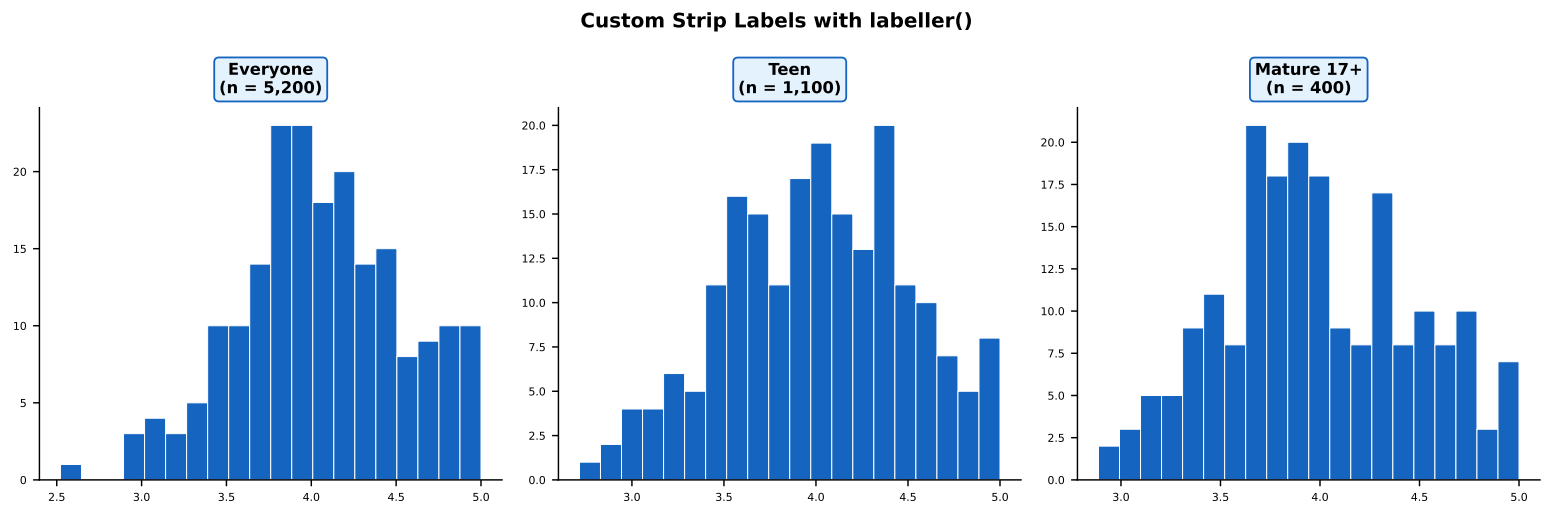
# Facet + Geom Combinations

# Facet + Smooth: Per-Panel Trends

**Data Insight**

When `geom_smooth()` is combined with `facet_wrap()`, the smoother is fitted independently within each panel. This reveals group-specific trends that would be averaged out in a single-panel, colour-coded plot.

# Custom Strip Labels




**Pro-Tip**

Use `labeller = label_both` to show "Variable: Level" or a custom labeller function to add sample sizes. In matplotlib, use `ax.set_title(f"{name} (n={count})")`.

# Facet vs Patchwork

# When to Facet vs When to Compose

**Facet vs Patchwork/GridSpec: Decision Guide**

| Situation | Tool | Example |
|---|---|---|
| Same geom, same aes, different group | `facet_wrap / facet_grid` | Comparing same measure across groups |
| Different geom or different aes | `patchwork / GridSpec` | Dashboard: bar + scatter + histogram + boxplot |
| Same data, different chart type | `patchwork / GridSpec` | Showing same data from multiple perspectives |

# Decision Rule in Code

```r
# FACET: Same geom, same aes, different group
ggplot(df, aes(x = month, y = revenue)) +
  geom_line() +
  facet_wrap(~region, ncol = 3)

# PATCHWORK: Different geom or different aes per panel
library(patchwork)
p_bar <- ggplot(df, aes(x = cat, y = n)) + geom_col()
p_hist <- ggplot(df, aes(x = rating)) + geom_histogram()
p_scatter <- ggplot(df, aes(x = reviews, y = rating)) + geom_point()
p_box <- ggplot(df, aes(x = type, y = rating)) + geom_boxplot()

# Compose: | = side by side, / = stack
(p_bar | p_hist) / (p_scatter | p_box) +
  plot_annotation(tag_levels = "a")
```

# Synthesis

# Faceting Decision Checklist

1. **How many groups?** 1–3: colour. 4+: facet.
2. **How many faceting variables?** 1: `facet_wrap`. 2: `facet_grid`.
3. **Comparable ranges?** Yes: `scales = "fixed"`. No: `scales = "free"` + warning label.
4. **Same geom across panels?** Yes: facet. No: patchwork / GridSpec.
5. **Need independent smoothers?** Yes: facet + `geom_smooth()` fits per panel automatically.
6. **Need context in strip labels?** Use `labeller = label_both` or custom function.

# Key Takeaways

1. **facet_wrap(~var)** splits by one variable into a wrapped grid; **facet_grid(row~col)** splits by two variables into a matrix.
2. **Fixed scales** (default) enable fair comparison; **free scales** reveal within-group detail at the cost of cross-panel comparability.
3. Faceting produces Tufte's **small multiples** automatically: same chart, same axes, different data subset per panel.
4. **Colour** for $\leq 3$ groups; **facet** for $4+$.
5. **Facet** when all panels use the same geom; **patchwork/GridSpec** when panels use different geoms.
6. In Python: `plotnine.facet_wrap`, `sns.FacetGrid`, or `plt.subplots` with a loop.

# Essential Readings

- Wickham, H. (2016). *ggplot2*, Chapter 7 (Facets).
- Tufte, E. R. (1983). *The Visual Display of Quantitative Information*, pp. 170–174 (Small Multiples).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 21 (Multi-Panel Figures).
- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapter 9.5 (Facets).

**Next Module**: Themes & Publication-Quality Styling (W02-M06)